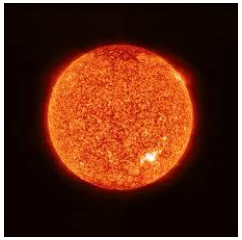
# Data,Variable & Object

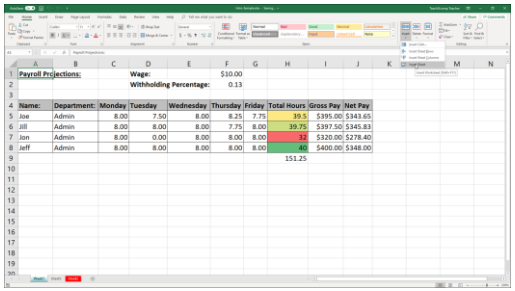
Professor: Tsungnan-Lin

# What is data?

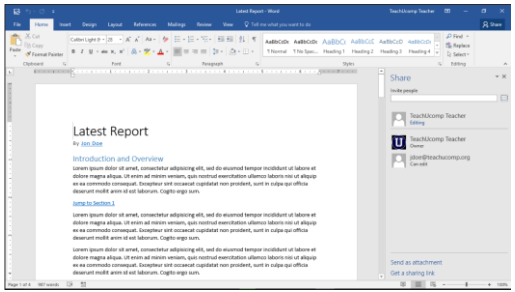
Human might think of data as:

- images
- videos
- excel sheets
- word documents
- powerpoint slides
- etc.

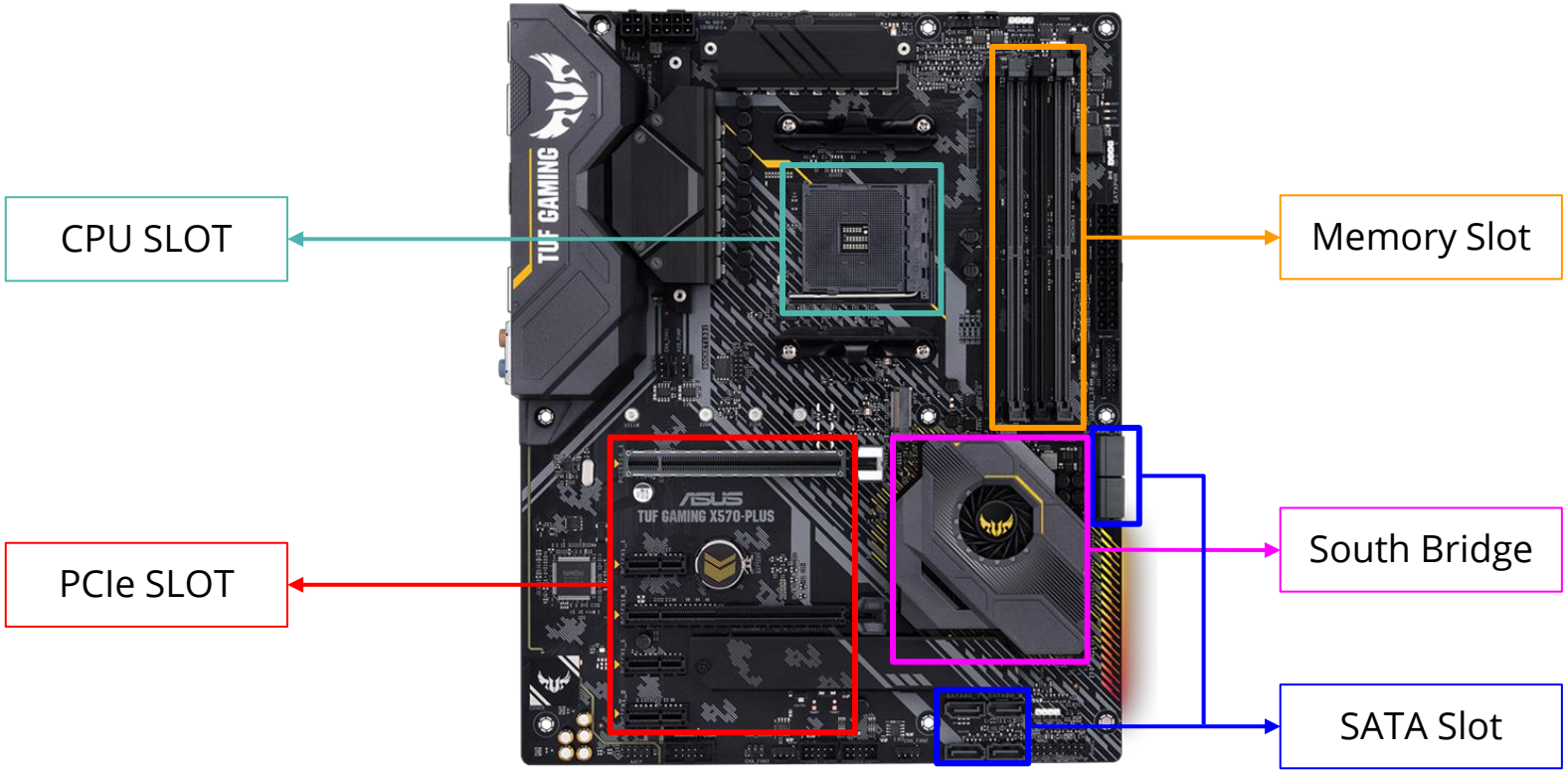


| | A | B | C | D | E | F | G | H | I | J | K | L | M | N |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Payroll Projections: | | | Wage: | | $10.00 | | | | | | | | |
| 2 | | | | Withholding Percentage: | | 0.13 | | | | | | | | |
| 3 | | | | | | | | | | | | | | |
| 4 | Name: | Department: | Monday | Tuesday | Wednesday | Thursday | Friday | Total Hours | Gross Pay | Net Pay | | | | |
| 5 | Joe | Admin | 8.00 | 7.50 | 8.00 | 8.25 | 7.75 | 39.5 | $395.00 | $343.65 | | | | |
| 6 | Jill | Admin | 8.00 | 8.00 | 8.00 | 7.75 | 8.00 | 39.75 | $397.50 | $345.83 | | | | |
| 7 | Jon | Admin | 8.00 | 0.00 | 8.00 | 8.00 | 8.00 | 32 | $320.00 | $278.40 | | | | |
| 8 | Jeff | Admin | 8.00 | 8.00 | 8.00 | 8.00 | 8.00 | 40 | $400.00 | $348.00 | | | | |
| 9 | | | | | | | | 151.25 | | | | | | |
| 10 | | | | | | | | | | | | | | |
| 11 | | | | | | | | | | | | | | |
| 12 | | | | | | | | | | | | | | |
| 13 | | | | | | | | | | | | | | |
| 14 | | | | | | | | | | | | | | |
| 15 | | | | | | | | | | | | | | |
| 16 | | | | | | | | | | | | | | |
| 17 | | | | | | | | | | | | | | |
| 18 | | | | | | | | | | | | | | |
| 19 | | | | | | | | | | | | | | |



# Recap - Hardware

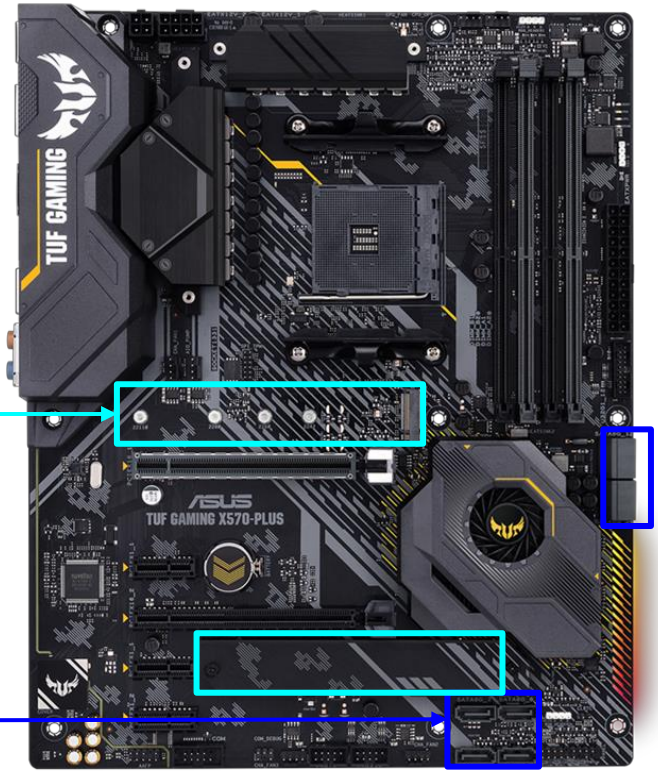


# Recap - Hardware

"Data" is usually stored in storage devices known as "**hard drives**."

Ex:

- Hard disk drive (HDD)
- Solid-state drive (SSD)

# What can computers do?

- Math computing -> We need numbers
  - x = 3, Data type for x is Integer.
  - y = 3.14, Data type for y is Float.
- Word processing -> We need characters
  - s = ‘Hello world’, Data type for s is String.

# How data stored in Computer?

Computer considers data as a stream of binary value called bit (0 or 1).

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 1 | 1 | 1 | 0 |

- **8 bits** equals to **1 byte**
- **2 bytes** equals to **1 word**

**Usually, we think of data with the unit of byte.**

# How data stored in Computer?

- ASCII code :

| DEC | OCT | HEX | Symbol | Description |
|---|---|---|---|---|
| 65 | 101 | 01000001 | A | Uppercase A |
| 66 | 102 | 01000010 | B | Uppercase B |
| 67 | 103 | 01000011 | C | Uppercase C |
| 68 | 104 | 01000100 | D | Uppercase D |

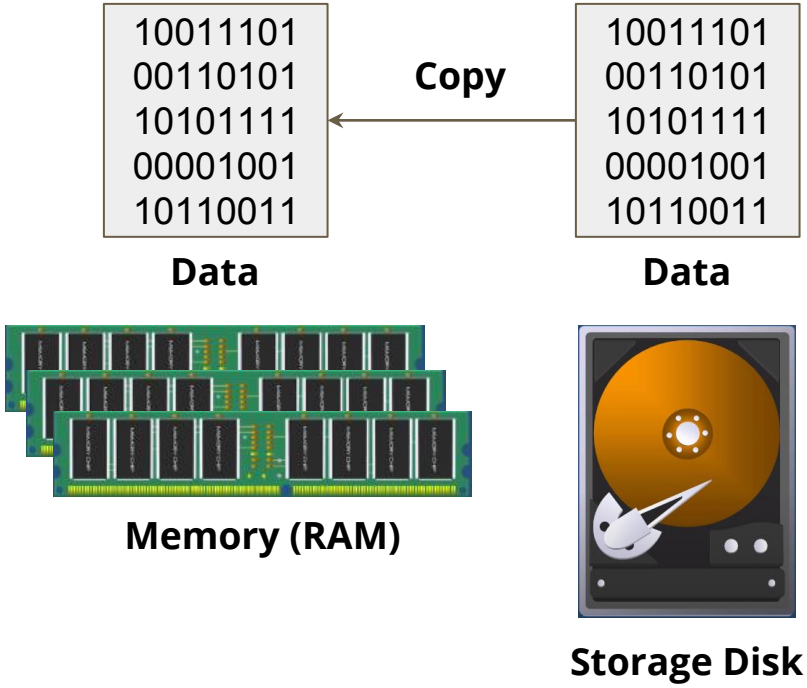
# How data stored in Computer?

- More bits cause more information and more usage in memory
  - For a three bits data:
    000 – 111: present at most 8 kinds of value, occupy 3 bits in memory.
  - For a five bits data:
    00000 – 11111: present at most 32 kinds of value, occupy 5 bits in memory.

# Where is data

Data will only exist in either **storage disk** or **memory**.

Whenever we want to process data, **we load data into memory from stroage disk**.



# What is Variable

A variable holds a value. You can change the value of a variable at any point of your program by an Assignment statement. (more will be introduce later…)

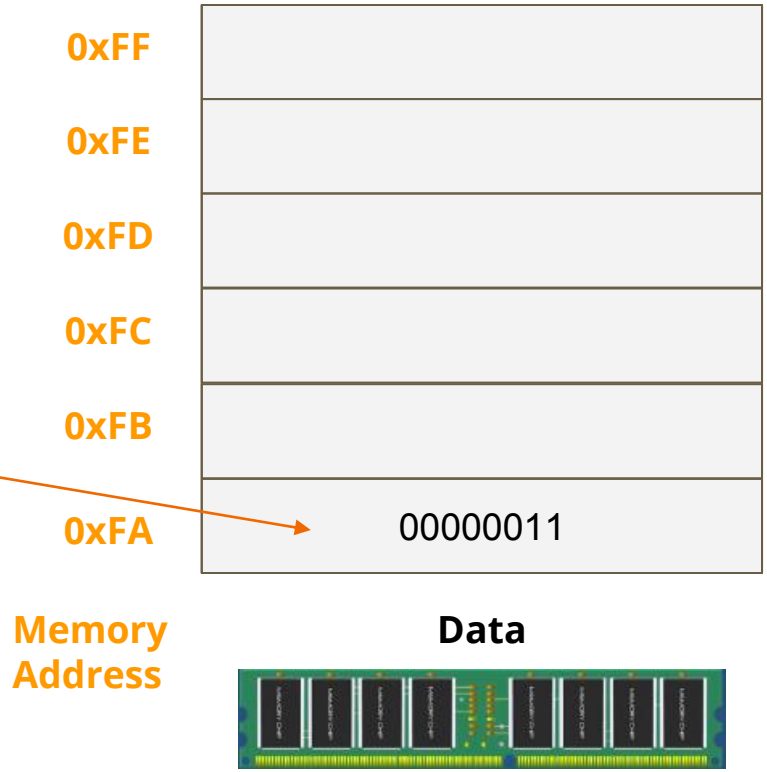
Ex: x=3

‘=‘ means assignment.

# Data in memory

Example: when we type x = 3 in python, it means computer allocate spaces in memory for storing 3.

x = 3

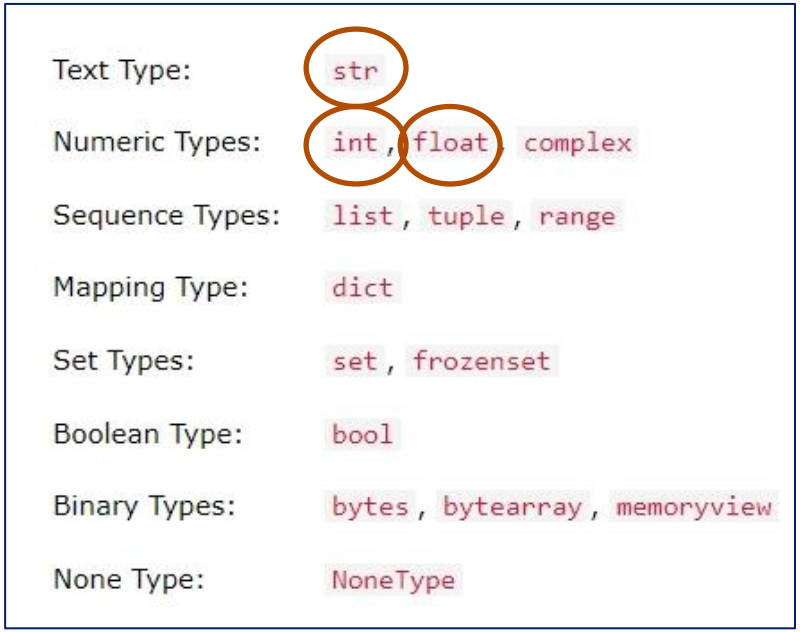


# Data Type

- Built-in Data Types

Variables can store data of different types, and different types can do different things.

Python has the following data types built-in by default, in these categories:

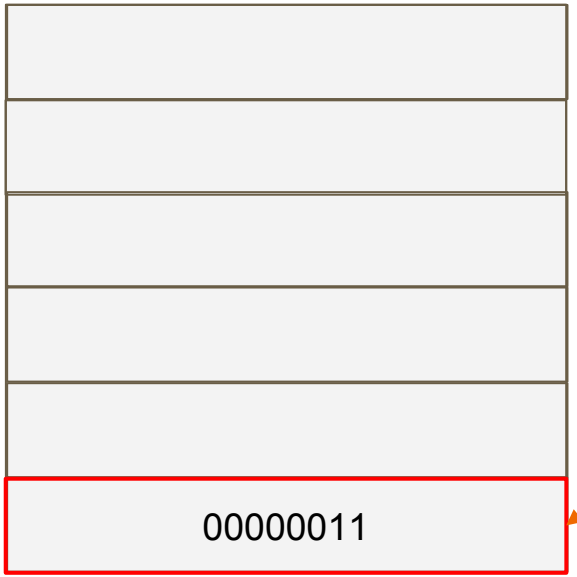


# Preview: Assignment Operator

- To assign values to variables.

| operator | Description | Example |
|---|---|---|
| = | Assign right value to left variable. | a = b |
| += | Subtraction | a += b |
| *= | Multiplication | a *= b |
| /= | Division | a /= b |
| %= | Modulus | a %= b |
| //= | Floor Division | a //= b |

# Data Type (Integer number)

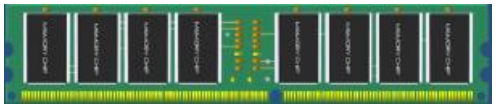



```
x = 3
print(type(x))
```

```
<class 'int'>
```

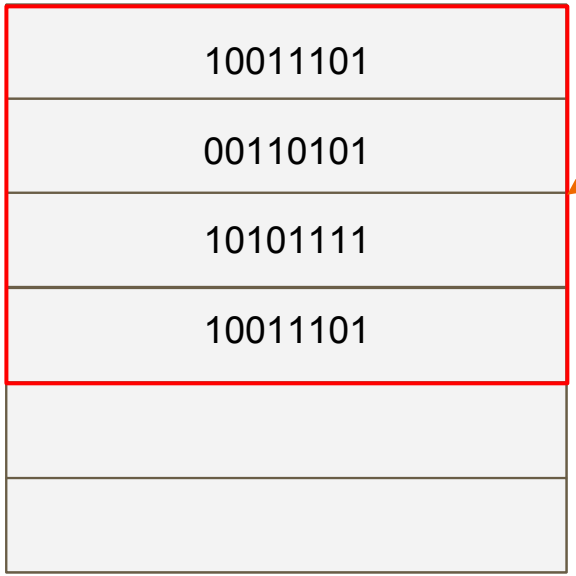
Memory Address

**Data**



Integers

# Data Type (Floating point number)



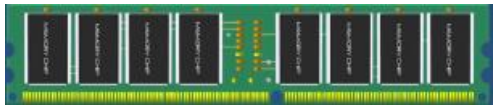

y = -4.17391338516e-21

```
y = -4.17391338516e-21
print(type(y))
```

```
<class 'float'>
```

# Data Type (String)

| Memory Address | Data |
| --- | --- |
| 0xFF | 1101110 (n) |
| 0xFE | 1101111 (o) |
| 0xFD | 1101000 (h) |
| 0xFC | 1110100 (t) |
| 0xFB | 1111001 (y) |
| 0xFA | 1110000 (p) |



z = ‘python’

```
z = 'python'
print(type(z))
```

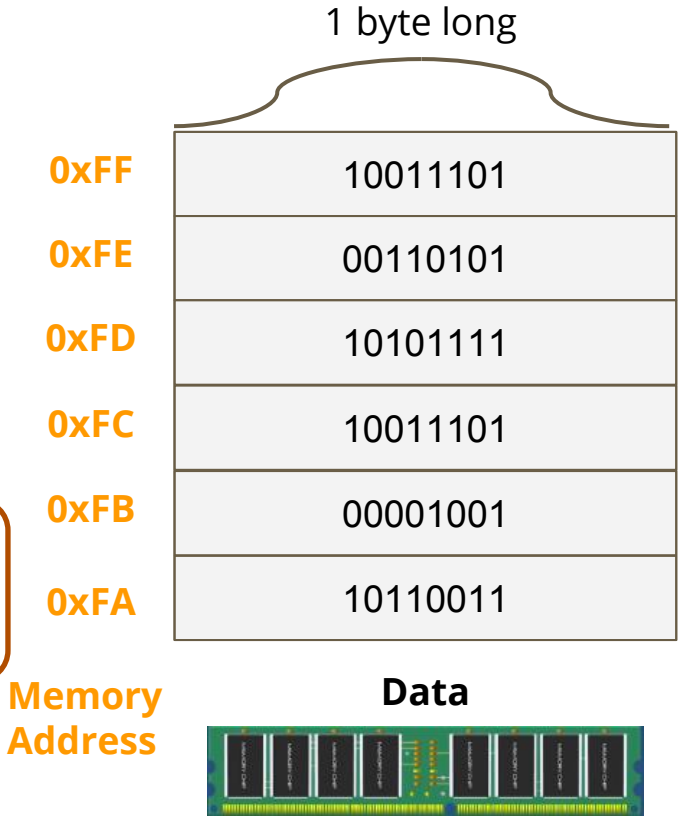
```
<class 'str'>
```

# Overflow

Each memory cell can only contain one-byte information.

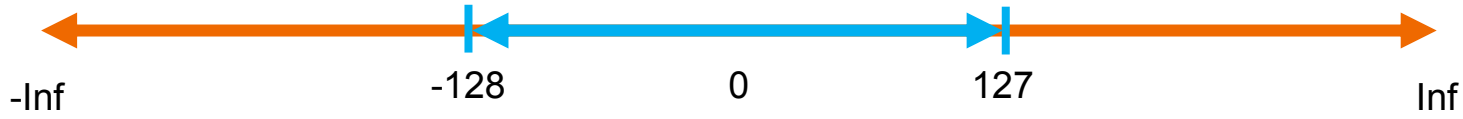
For one byte, the value range of integer is -128 to 127. (**00000000** - **11111111**)

Overflow:
100 + 100 will become -55 rather than 200



# Overflow

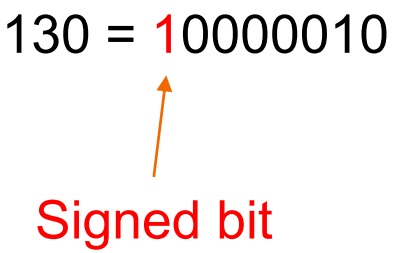
- Ex: 1 byte




For one byte, the value range of integer is -128 to 127.
(00000000 - 11111111)

# Overflow Example

Ex: 1 byte for 130




So it will become -126 (2's complement)

# Underflow

We need to allocate space for fractional part of number. If the data is too small to be represented, it will be discarded, which is called underflow.

Ex:
The data is only valid until the fourth decimal place. If there is a number 0.00001, the fifth decimal place needs to be discarded, which is an underflow.

# What is variable

In computer science, variable is just a nickname for the data variable refer to.

You need to explicitly or implicitly specify the type of variable so that computer knows how to interpret the byte stream.

**In python, you don't need to specify the type of variable. The python runtime automatically determine the type of variable.**

**Python is a dynamic language**

# Dynamic

- Dynamic **Type**
  - Don’t need to specify the type of variable. Python runtime automatically determine the type of variable.

- Dynamic **Memory size**
  - Don’t need to handle memory allocation, Python will help us do this.
    - Ex: 12.5 ^ 100 in Python v.s 12.5 ^ 100 in C.

# Dynamic Type

- Static type (C)

```c
#include <stdio.h>

int main() {
    int b = 99;

    b = "hello";

    return 0;
}
```

```
❯ gcc test.c
test.c:6:7: warning: incompatible pointer to integer conversion assigning to 'int' from 'char[6]' [-Wint-conversion]
    b = "hello";  // 這將導致編譯錯誤
      ^ ~~~~~~~
```

- Dynamic type (Python)

```
[13]
a = 1
b = 'hello'
print(b)
b = 99
print(b)
```

```
hello
99
```

# Dynamic size

- c/c++ for counting 100!
- We need to dynamically maintain the size of the big number.

- A bunch of work

- Python for counting 100!
- Python will help us dynamically allocate the memory for big numbers.

```
import math
print(math.factorial(100)) # 100!
```

# Declare variable in Python

In [2]:
```
a = 10
print(type(a))
b = 3.14
print(type(b))
s = "python"
print(type(s))
```

integer

Floating point

String

executed in 7ms, finished 10:05:46 2021-09-26

```
<class 'int'>
<class 'float'>
<class 'str'>
```

Use **type()** to output type of variable.

# Declare variable

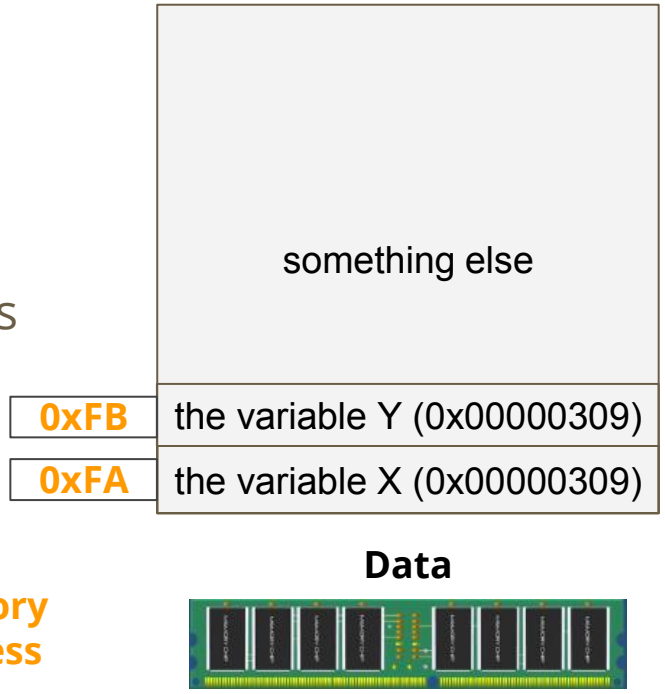
- Example:

```
In [21]: X = 777
         Y = 777
         print(X is Y)
         print(X == Y)
```

executed in 10ms, fir

```
False
True
```

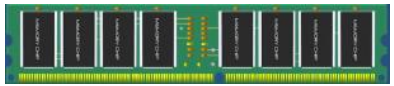
Compare address
Compare value



# Why variable

Variable lets human easliy refer to data with friendly and meaningful name.

| | Memory Address | Data |
|---|---|---|
| A | 0xFF | 10011101 |
| B | 0xFE | 00110101 |
| | 0xFD | 10101111 |
| | 0xFC | 10011101 |
| | 0xFB | 00001001 |
| | 0xFA | 10110011 |



Suppose you want to add num1 & num2 together...



# What is Object

In natural, If the thing can be decribed by some feature and some abilities, it is an Object.

- Object:
  - Attribute
    - To decribe what it is
  - Method
    - To discribe what it can do

# Variable vs object

- Variable
  - Memory allocate a space for storing data.
- Object
  - Memory allocate a space for storing data.
  - Exist **method** for us to use.

# What is Object

```
In [3]: s = "python"
        dir(s)
```

executed in 32ms, finished 10:29:13 2021-09-26

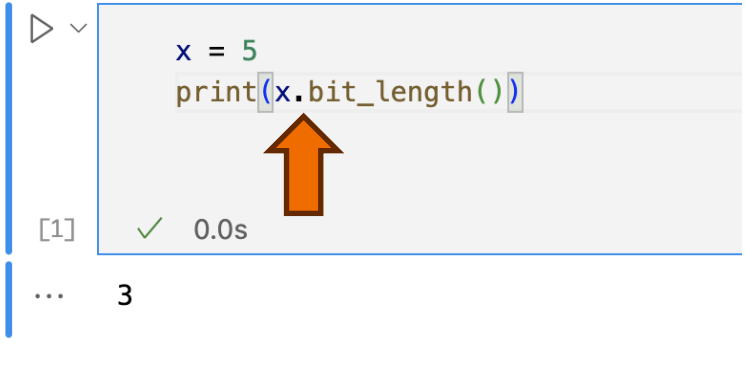
```
Out[3]: ['__add__',
         '__class__',

         ...

        'translate',
        'upper',
        'zfill']
```

Use dir() to output object's attributes.

# Add-on: Object Orientation in Python

**In the world of Python, everything is an object**. This includes everything from the most basic data types to functions, modules, and even instances of exceptions created at runtime.

For example, even a simple integer has built-in methods that you can use to perform various operations:



# Add-on: Comparison with C Language

In contrast to Python, C language is not an object-oriented language. In C, basic data types such as int, char, etc., are **just primitive data types and do not possess the attributes or methods expected of objects**.

For example, in C, you cannot directly call methods on an integer because it is merely a numerical representation and lacks additional object functionalities:

```c
#include <stdio.h>

int main() {
    int x = 5;
    printf("%d", x);

    return 0;
}
```

# What is Object

Get attribute: **<object>.<attribute>**

```
In [4]: s = "python"
        s.__class__
```

executed in 9ms, finished 10:35:17 2021-09-26

```
Out[4]: str
```

Use method: **<object>.<method>()**

```
In [5]: s = "python"
        s.upper()
```

executed in 8ms, finished 10:35:34 2021-09-26

```
Out[5]: 'PYTHON'
```

# Help function

```
In [12]: help(y.upper)
```

executed in 6ms, finished 10:48:38 2021-09-26

```
Help on built-in function upper:

upper(...) method of builtins.str instance
    S.upper() -> str

    Return a copy of S converted to uppercase.
```

Use **help()** to watch the description.

# Object for Python

- An object is a fundamental building block of an object-oriented language.
- An object must have its type.
- The types mentioned so far are all **built-in data types** of programming languages (for example: int, string, list, set, etc.).
- Once create an object, computer must allocate memory space for it.
- **In Python, everything is an “object”.**

# Object for Python - Example

```
In [7]: x = 777
        y = 'bbb'
        print(hex(id(x)))
        print(hex(id(y)))
```

executed in 17ms, finished 10:

```
0x7f3f55ff1150
0x7f3f8435f260
```

```
In [11]: print(type(x))
         print(type(y))
```

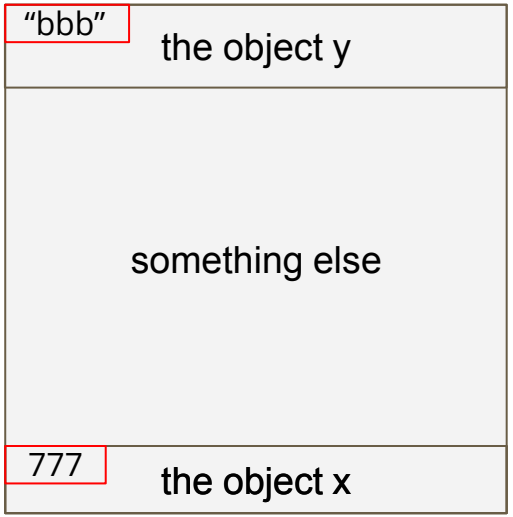
executed in 8ms, fin

```
<class 'int'>
<class 'str'>
```

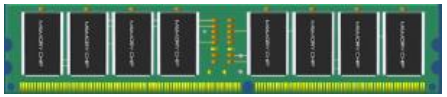
0x7f3f8435f260

0x7f3f55ff1150

Memory Address




Data



# Different Types, different attributes

```
In [9]: dir(x)
```

executed in 14ms, f

```
Out[9]: ['__abs__',
         '__add__',
         '__and__',
         ...
         'imag',
         'numerator',
         'real',
         'to_bytes']
```

```
In [10]: dir(y)
```

executed in 17ms, finish

```
Out[10]: ['__add__',
          '__class__',
          '__contains__',
          ...
          'title',
          'translate',
          'upper',
          'zfill']
```

# Everything is an object !

Build-in functions are objects, too:

```
In [15]: type(print)
         executed in 13ms, finished 10:50:37 2021-09-26
Out[15]: builtin_function_or_method
```

```
In [13]: dir(print)
         executed in 15ms, finished 10:50:23 2021-09-26
Out[13]: ['__call__',
          '__class__',
          ...
          '__str__',
          '__subclasshook__',
          '__text_signature__']
```

# Everything is an object !

Modules are objects, too:

```
In [1]: import math
        type(math)

Out[1]: module

In [2]: print(dir(math))

['__doc__', '__loader__', '__name__', '__package__', '__spe
c__', 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'a
tanh', 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degree
s', 'dist', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs', 'fa
ctorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd',
'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'i
sqrt', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2',
'modf', 'nan', 'perm', 'pi', 'pow', 'prod', 'radians', 'rem
ainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trun
c']
```

What is the type of type() ?

# Useful function Recap

These functions may help you:

| function | Description | example |
|---|---|---|
| dir() | list out attributes for the object. | dir(<object>) |
| help() | watch the description for the object. | help(<object>), help(<object>.<attribute>) |
| id() | get the address of the object. | id(<object>) |
| print() | output the value of the object. | print(<object>) |
| type() | output the type of the object. | type(<object>) |

# Operator

Professor: Tsungnan-Lin

# Arithmetic Operators

- Used to perform mathematical operations like addition, subtraction, multiplication and division.

| operator | Description | Example |
|---|---|---|
| + | Addition | a + b |
| - | Subtraction | a - b |
| * | Multiplication | a * b |
| / | Division | a / b |
| % | Modulus | a %b |
| // | Floor Division | a // b |

# Assignment Operator

- To assign values to variables.

| operator | Description | Example |
|---|---|---|
| = | Assign right value to left variable. | a = b |
| += | Subtraction | a += b |
| *= | Multiplication | a *= b |
| /= | Division | a /= b |
| %= | Modulus | a %= b |
| //= | Floor Division | a //= b |

# Comparison Operators

- Compares the values of two operands and returns True or False based on whether the condition is met.

| operator | Description | Example |
|---|---|---|
| == | Equal | a == b |
| != | Not Equal | a != b |
| > | Greater Than | a > b |
| < | Less Than | a < b |
| <= | Less Than or Equal To | a <= b |
| >= | Greater Than or Equal To | a >= b |

Please type == not = !

# Logical Operators

- Used to perform logical operations on the values of variables.

| operator | Description | Example |
|---|---|---|
| and | returns True if both the operands are True | a and b |
| or | returns True if either of the operand is True | a or b |
| not | returns False if either of the operand is True<br>returns True if either of the operand is False | not b |

# Bitwise Operators

- Used to manipulate individual bits.

| operator | Description | Example |
|---|---|---|
| & | Bitwise AND | a & b |
| \| | Bitwise OR | a \| b |
| ^ | Bitwise XOR | a ^ b |
| ~ | Bitwise NOT | ~a |
| << | Bitwise left shift | a << n |
| >> | Bitwise right shift | a >> n |

# Identity operators

Used to check if two variables are located on the same part of the memory.

| operator | Description | Example |
| --- | --- | --- |
| is | True if the operands are refer to the same object | a is b |

# Dot operators

Used to call object's attribute.

| operator | Description | Example |
| --- | --- | --- |
| . | Call object's attribute. | a = 'string'<br>a.upper() |

# Order of operations

- Parentheses > Exponentiation > Multiplication and Division > Addition and Subtraction

- Association:
  - In general, left to right
  - For exponentiation, from right to left

```
In [32]:
print(4 ** 3 **2) # not 4096
```

```
262144
```

```
In [1]:
# try out
x = 5 + 6 * 3 ** 2 - 2
y = (5 + 6) * 3 ** 2 - 2
z = (5 + 6 * 3) ** 2 - 2
print('x =',x)
print('y =',y)
print('z =',z)
```

```
x = 57
y = 97
z = 527
```

# Polymorphism

- In python, operator act as different functionality when meets different operands.
- Ex:
  - 1 + 1 =2
  - ‘ab’ + ‘cd’ = ‘abcd’
- This is a property of polymorphism.

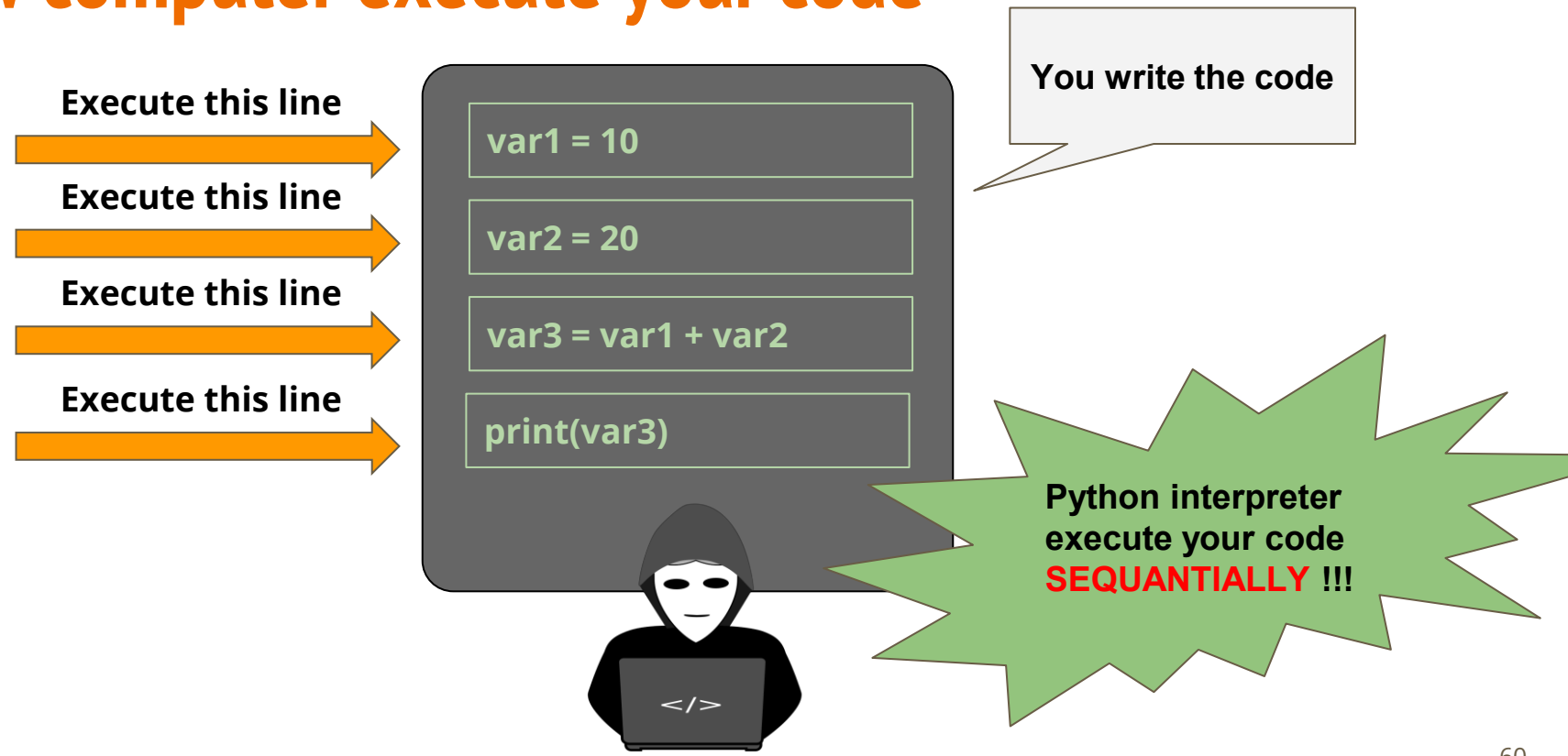
# Polymorphism

- In python, operator act as different functionality when meets different operands.
- Ex:
  - 1 + 1 =2
  - ‘ab’ + ‘cd’ = ‘abcd’
- This is a property of polymorphism.


# Control Structure

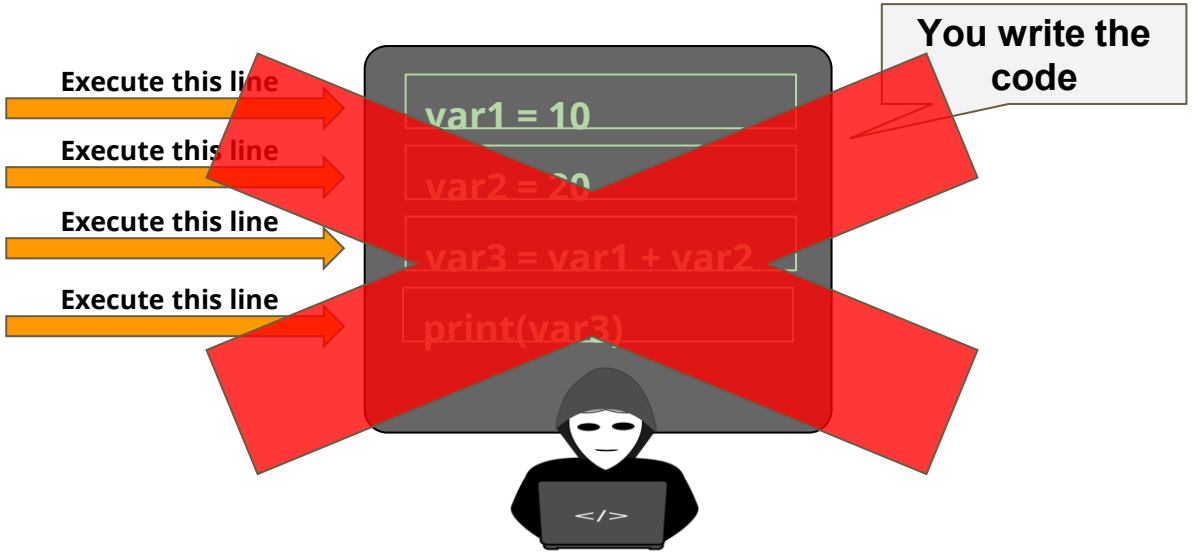
Professor: Tsungnan-Lin

# How computer execute your code



# How computer execute your code

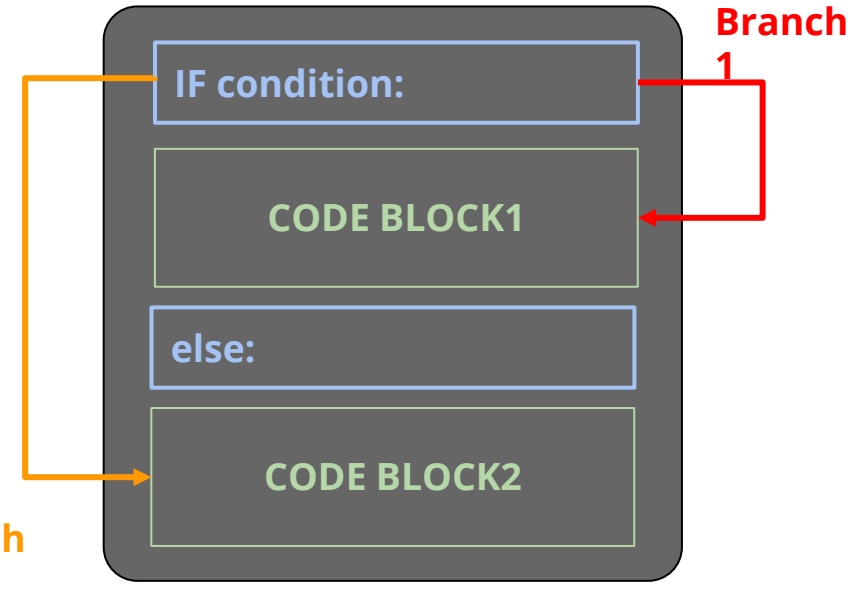
**What if you want to choose one of the variables from var1 and var2 and print the larger one ?**



# Control statement

Every programming language provides you with control statements, such as **IF**, **ELSE**, **ELIF**, and etc.

If condition, a **Boolean expression**, is evaluated to be **True**, the Code Block 1 will be executed; otherwise, Code Block 2 will be executed.



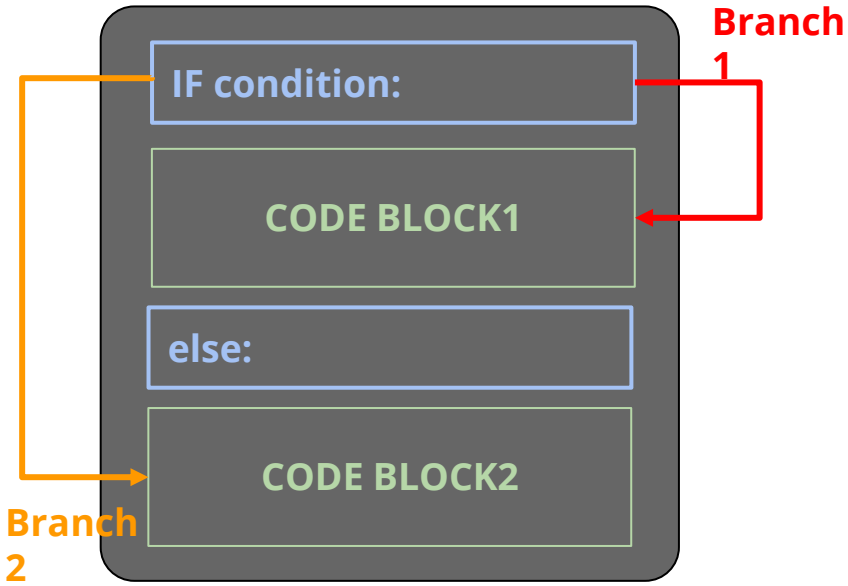
# Control statement example

We use tab as Code Block Indentation in Python.

```
In [1]: a = 7
        if a > 5:
            print('bigger than 5')
        else:
            print('smaller or equal than 5')

        bigger than 5
```

Block 1

Block2



# Chained conditionals

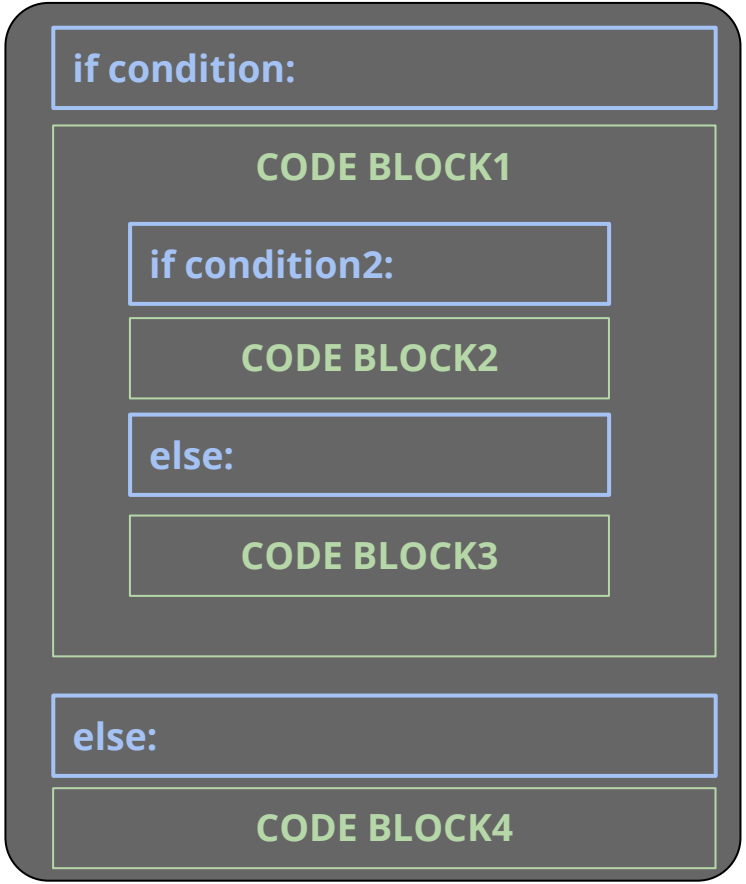
In [1]:
```
x = 3
y = 4
if x < y:
    print('x is less than y')
elif x > y:
    print('x is greater than y')
else:
    print('x and y are equal')
```

```
x is less than y
```

# Nested conditionals

In [6]:
```
x = 2
if x >= 0:
    if x < 10:
        print('x is a single-digit number.')
    else:
        print('x larger or equal than 10')
else:
    print('x is negative')
```

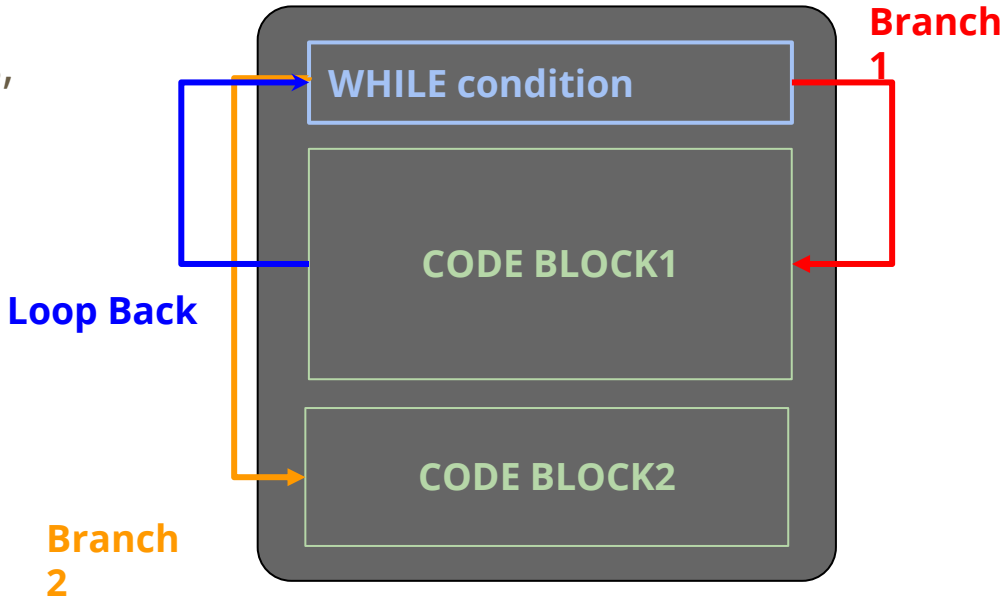
```
x is a single-digit number.
```



# Loop statement

Every programming language provides you with loop statements, such as **FOR**, **WHILE**, and etc.

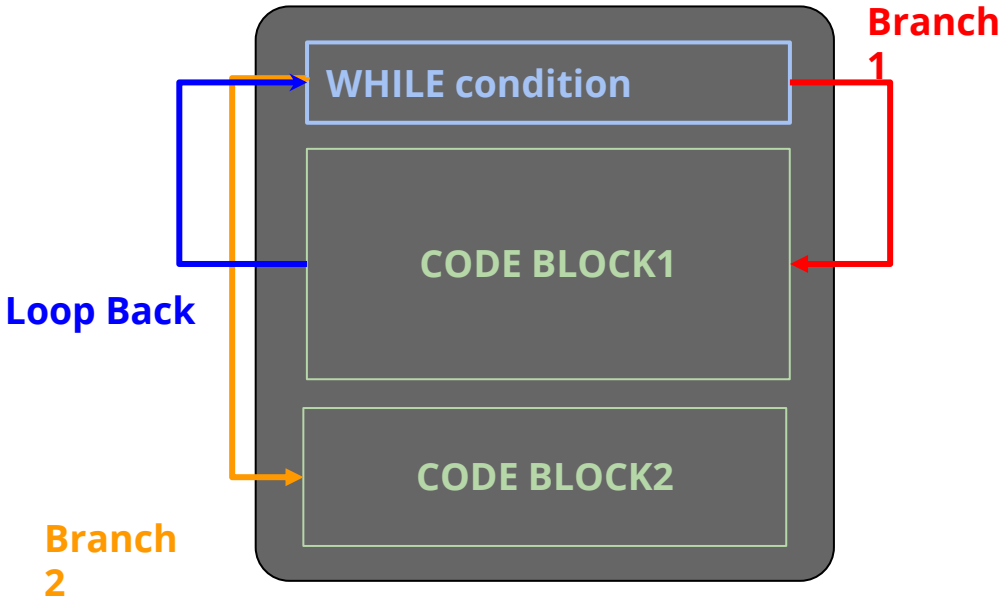
These statements help you **repeatedly** do some tasks.



# Loop statement example (Count 1 to 10)

```
In [2]: i = 1
        while i < 11:
            print(i)
            i += 1
```

```
1
2
3
4
5
6
7
8
9
10
```

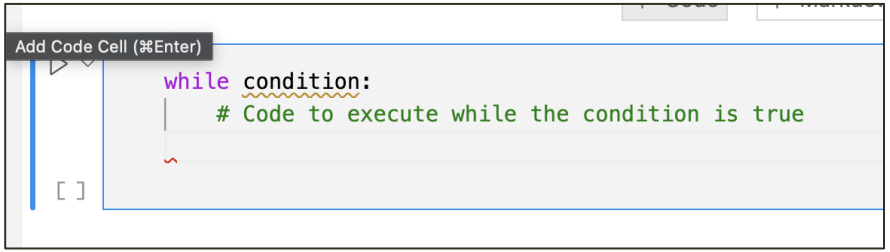


# While Loops in C vs. Python

- C Language
- In C, the while loop uses curly braces { } to define the block of code that should be repeated. Here is a basic example:

```
 9      while (condition) {
10      // Code to execute while the condition is true
11  }
12
```

- Python
- In Python, indentation is used instead of braces to define the blocks of code.



# Break and continue statements

- break : Stop the loop even if the while condition is true

In [7]:
```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

```
1
2
3
```

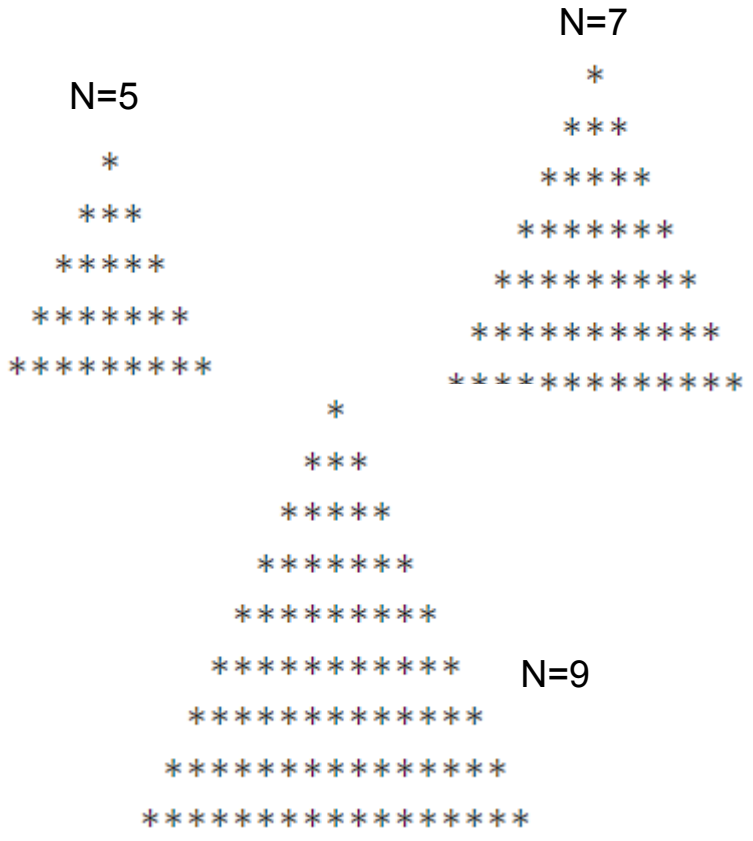
- continue : Stop the curr iteration, and continue with the next

In [8]:
```
i = 1
while i < 6:
    i += 1 # what if put this to the end of the loop?
    if i == 3:
        continue
    print(i)
```

```
2
4
5
6
```

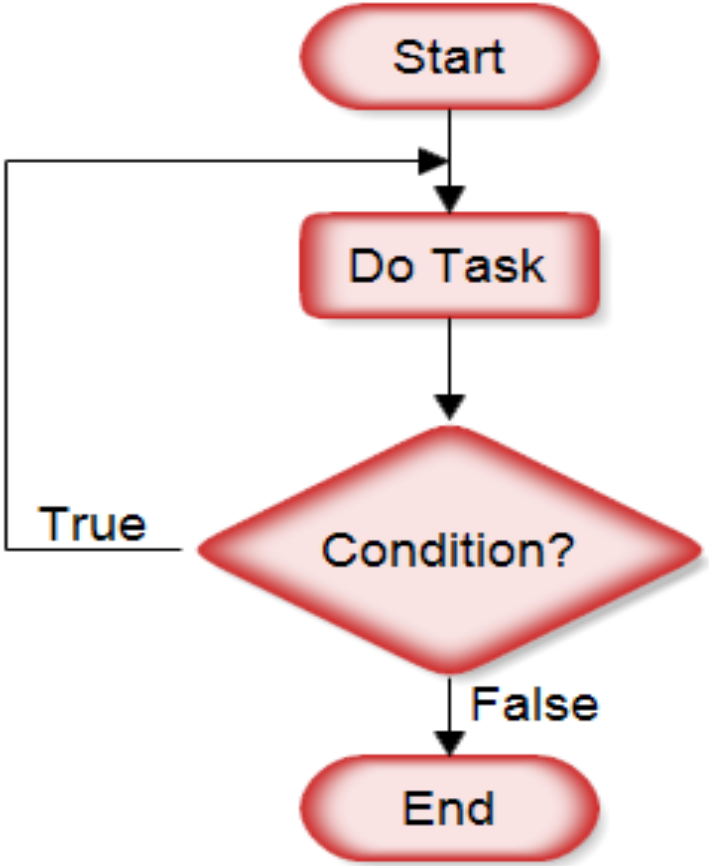
# Exercise: print a pyramid

- Use while loop to print out the pyramid on the right.



# Do While Loop

There is no do while loop in python!




# While Loop